



UNIVERSITY OF SCIENCE - VNUHCM

FACULTY OF INFORMATION TECHNOLOGY

SOFTWARE TESTING

HW7 - API TESTING

Software Testing Project Report

Authors:

Lưu Thanh Thuý

(22127410)

ltthuy22@clc.fitus.edu.vn

Supervisors:

Teacher Trần Duy Hoàng

Teacher Hồ Tuấn Thanh

Teacher Trương Phước Lộc

August 24, 2025

Table of Contents

1	Group Information	1
2	Introduction	1
3	Test Environment	1
4	Methodology	2
5	API 1 – Sign in	2
5.1	Test Cases	2
5.2	Execution & Evidence	3
5.3	Bugs Found	3
6	API 2 – Add User	3
6.1	Test Cases	3
6.2	Execution & Evidence	3
6.3	Bugs Found	4
7	API 3 – Edit User	4
7.1	Test Cases	4
7.2	Execution & Evidence	4
8	CI Integration — Step-by-Step (with Visuals)	4
8.1	Preparation	4
8.2	Trigger Events	5
8.3	Workflow Steps	5
8.3.1	Open the Actions Tab	5
8.3.2	Checkout Code	5
8.3.3	Set Up Node.js	5
8.3.4	Install Newman & Reporter	5
8.3.5	Run Postman Collections	6
8.3.6	Monitor Workflow Logs	6
8.3.7	View Newman HTML Reports	7
8.3.8	Download Reports from Artifacts	7
8.4	Output	7
8.5	Benefits of CI Integration	8
9	AI / LLM Support & Prompts	8
9.1	Tool Used	8
9.2	How AI was Applied	8
9.3	Prompts Used	8

1 Group Information

Group ID: 07

Member Name	Student ID	Assigned Features	Status
Cao Uy��n Nhi	22127310	Payment Brands Favourite	Done
L��u Thanh Thu��y	22127410	Sign In Add User Edit User	Done
Nguy��n Ph���c Minh Tr��	22127424	Homepage - Sort - Search	Done
V�� L�� Vi��t T��	22127435	Sign In - Invoice	Done
Tr��n Th��i C��t T����ng	22127444	Send Contact Create Category Edit Category	Done

2 Introduction

- **Project under test:** The Toolshop (sprint5-with-bugs)
- **Scope:** Functional and security testing of 3 APIs: **Sign in**, **Add User**, and **Edit User**.
- **Objectives:**
 - Validate input handling, role-based access controls (RBAC), and business logic integrity.
 - Apply **Domain Testing** to test boundary conditions, input formats, and equivalence classes (e.g., email formats, password complexity, date of birth validation).
 - Implement **State Transition Testing** to verify state changes (e.g., login → token issuance → protected API access → logout → re-login) and admin actions like enabling/disabling users.
 - Identify security vulnerabilities such as SQL injection, cross-site scripting (XSS), RBAC bypass, brute-force attacks, and unauthorized access attempts.
- **Tools:** Postman for execution, GitHub Actions for CI, Markdown for documentation, Excel for test case management.

3 Test Environment

- **Base URL:** http://localhost:8091
- **Database:** Pre-seeded with default accounts for consistent testing:
 - Admin → admin@practicesoftwaretesting.com / welcome01
 - Customers → customer@practicesoftwaretesting.com / welcome01, etc.
- **Tools & Versions:**
 - Postman: v10.15 (or latest stable version at the time of testing)

- GitHub Actions: Hosted runners (Ubuntu-latest)
- **Platform:** Localhost environment running on a development machine (Windows/Linux/Mac) with sufficient resources. Tests were conducted against a local instance to ensure controlled conditions without external dependencies.

4 Methodology

- **Domain Testing:**
 - Tested boundary conditions and input formats for fields like email (valid/invalid formats, length, Unicode, aliases), password (length, complexity, equivalence to email), date of birth (future dates, underage users), and country/postcode formats (e.g., regex compliance).
 - Used equivalence partitioning to reduce redundant test cases while ensuring coverage of valid and invalid input ranges.
- **State Transition Testing:**
 - Verified state flows: Login → Token issuance → Access protected APIs → Logout → Re-login.
 - Tested admin-specific actions: Enabling/disabling user accounts and their impact on subsequent login attempts.
 - Validated RBAC: Ensured non-admin users cannot perform admin-only actions (e.g., user creation or modification).
- **Security Testing:**
 - Tested for common vulnerabilities: SQL injection, XSS, brute-force attacks, missing authentication headers, and attempts to manipulate API payloads (e.g., injecting unexpected fields or escalating roles).
 - Verified token expiration, reuse prevention, and rate-limiting mechanisms.

5 API 1 – Sign in

5.1 Test Cases

- **Total:** 30
- **File:** UC01_SignIn_TestCases.xlsx
- **Coverage:**
 - Happy path: Valid email and password combinations.
 - Invalid inputs: Empty email/password, malformed email (e.g., missing @, invalid TLD), weak passwords.
 - Edge cases: Email aliases (e.g., user+alias@domain.com), Unicode emails, case sensitivity.
 - Security: Brute-force attempts, rate-limiting enforcement, SQL injection, XSS payloads in inputs.
 - RBAC: Locked or suspended accounts, invalid token usage.
 - Token lifecycle: Expiration, reuse, and invalidation after logout.
 - Payload validation: Missing headers, malformed JSON, unexpected fields.

5.2 Execution & Evidence

- **Execution Method:** Tests executed via Postman Runner with assertions for expected status codes, response payloads, and error messages.
- **Evidence:**
 - Screenshots of Postman Runner results showing Pass/Fail outcomes for each test case.
 - Logs capturing request/response details, including headers and payloads.
 - Example issue: Login with an invalid email (e.g., user.domain.com) returned 200 OK with a token instead of the expected 422 Unprocessable Entity.

5.3 Bugs Found

- **BUG-001:** Invalid email format accepted.
 - *Description:* API accepts email without @.
 - *Expected:* 422 validation error.
 - *Actual:* 200 OK with token.
- **BUG-002:** Password returned in response.
 - *Description:* Response payload includes password field.
 - *Expected:* No sensitive fields in response.
 - *Actual:* Password is echoed.
- **BUG-003:** Missing brute-force protection.
 - *Description:* Unlimited failed attempts do not trigger logout.

6 API 2 – Add User

6.1 Test Cases

- **Total:** 30
- **File:** UC02_AddUser_TestCases.xlsx
- **Coverage:**
 - Happy path: Admin creates a new user with valid data.
 - Input validation: Duplicate emails (case-insensitive), invalid email/password/DOB/country.
 - Security: SQL injection and XSS payloads in fields (e.g., in name).
 - RBAC: Non-admin users attempting to create users, role escalation attempts (e.g., setting role=admin in payload).
 - Payload issues: Missing headers, malformed JSON, unexpected fields.
 - Audit logging: Verify that user creation events are logged.

6.2 Execution & Evidence

- **Execution Method:** Tests executed via Postman Runner with assertions for status codes, response validation, and audit log checks.
- **Evidence:**
 - Screenshots of request/response pairs for successful and failed attempts.
 - Example issue: Non-admin user calling /users/register returned 201 Created instead of 403 Forbidden.

6.3 Bugs Found

- **BUG-010:** Duplicate email not rejected.
- **BUG-011:** Non-admin can add user.
- **BUG-012:** Response leaks password.
- **BUG-013:** No audit log recorded for user creation.

7 API 3 – Edit User

7.1 Test Cases

- **Total:** 30
- **File:** UC03_EditUser_TestCases.xlsx
- **Coverage:**
 - Happy path: Update user fields (name, email, DOB, phone, postcode, enabled flag).
 - Input validation: Duplicate email checks, invalid DOB (future/underage), password complexity.
 - Admin actions: Disabling/enabling users, partial updates via PATCH requests.
 - Security: XSS and SQL injection payloads, RBAC enforcement (non-admin vs. admin access).
 - Payload issues: Malformed JSON, missing headers, unexpected fields.

7.2 Execution & Evidence

- **Execution Method:** Tests executed via Postman Runner with assertions for status codes and response validation.
- **Evidence:**
 - Screenshots of requests/responses, including attempts to set duplicate emails or future DOBs.
 - Example issue: Future DOB accepted without validation, returning 200 OK.

8 CI Integration — Step-by-Step (with Visuals)

Setting up Continuous Integration (CI) to run Postman collections using Newman on GitHub Actions. Follow each step below and refer to the screenshots for clarity.

8.1 Preparation

- /tests folder contains: UC01_SignIn.json, UC02_AddUser.json, UC03_EditUser.json.
- Environment file: toolshop-local.postman_environment.json (e.g., baseUrl = http://localhost:8091).
- Workflow file: .github/workflows/api-tests.yml.
- Newman + HTML reporter installed on the runner.

8.2 Trigger Events

The workflow runs automatically on:

- Push to the **main** branch.
- Pull requests targeting **main**.

8.3 Workflow Steps

8.3.1 Open the Actions Tab

- Go to your repository on GitHub → click **Actions**.
- Confirm a workflow named **api-tests** appears.

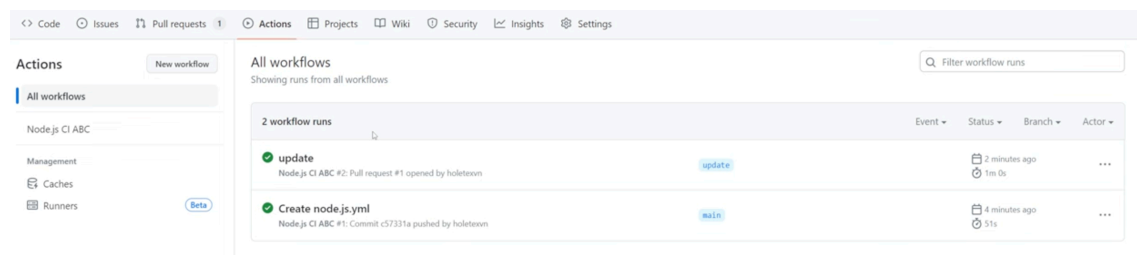


Figure 1: Github Actions

8.3.2 Checkout Code

Action

- The workflow runs `actions/checkout` to clone your repo onto the runner.

Expected

- Files under `/tests` and the workflow itself are available on the runner.

Why

- Newman will need to read the collections and environment directly from the workspace.

8.3.3 Set Up Node.js

The workflow installs **Node.js 16** using `'actions/setup-node`

8.3.4 Install Newman & Reporter

Install `newman` and `newman-reporter-html` globally.

Expected

- `newman -v` and HTML reporter are available in PATH.

Why

- You'll execute each collection and produce readable HTML reports.

8.3.5 Run Postman Collections

Action Run **each** collection with the shared environment:

- UC01_SignIn.json
- UC02_AddUser.json
- UC03_EditUser.json

Expected

- Console logs show request/response summaries.
- Exit code 0 if all assertions pass; non-zero if any fail.

Why

- CI should fail on regression to signal issues early.

8.3.6 Monitor Workflow Logs

View status and logs live in the **Actions** tab while the workflow runs.

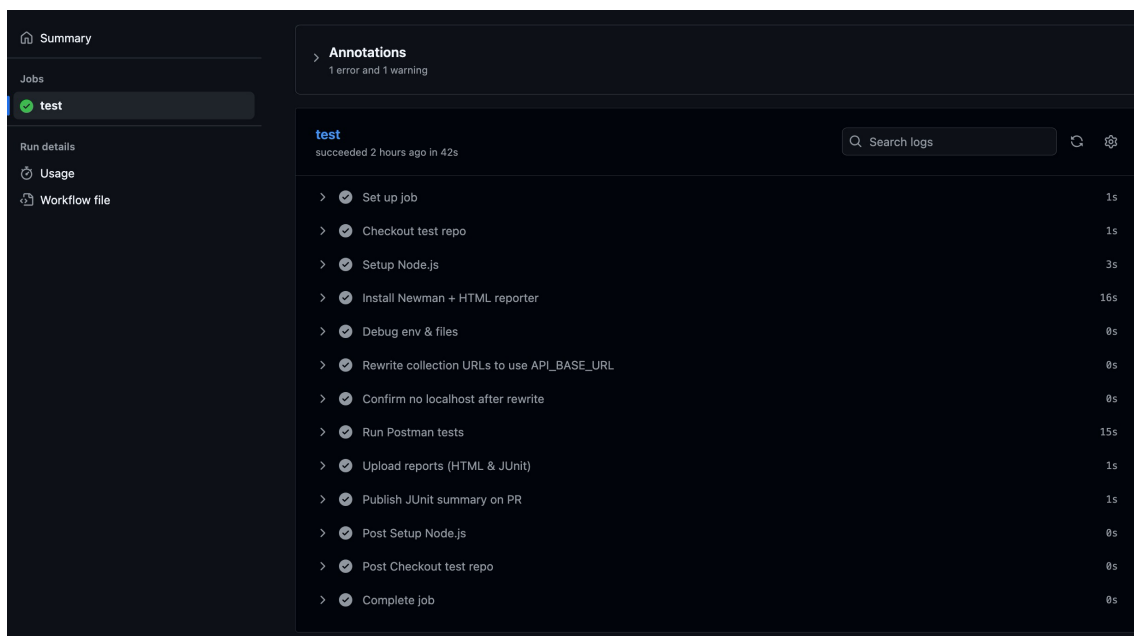


Figure 2: GitHub Actions Workflow Run

8.3.7 View Newman HTML Reports

Reports (`newman-report-*.html`) are generated in the `/reports` folder. These reports show detailed pass/fail breakdown and assertions.

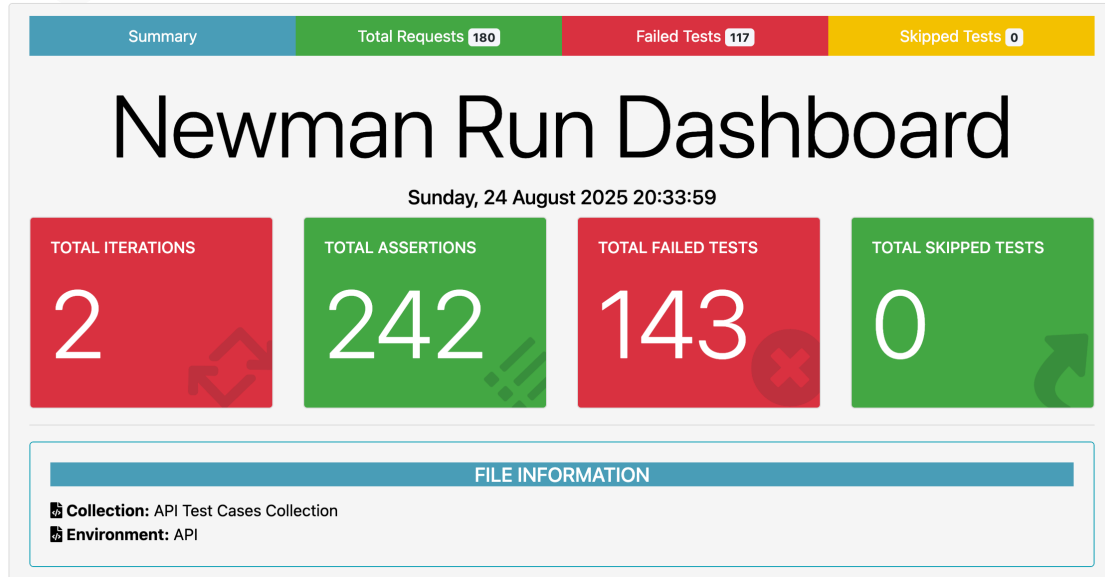


Figure 3: Newman HTML Report

8.3.8 Download Reports from Artifacts

Reports are uploaded as artifacts after the run. You can download them directly from the GitHub Actions interface.

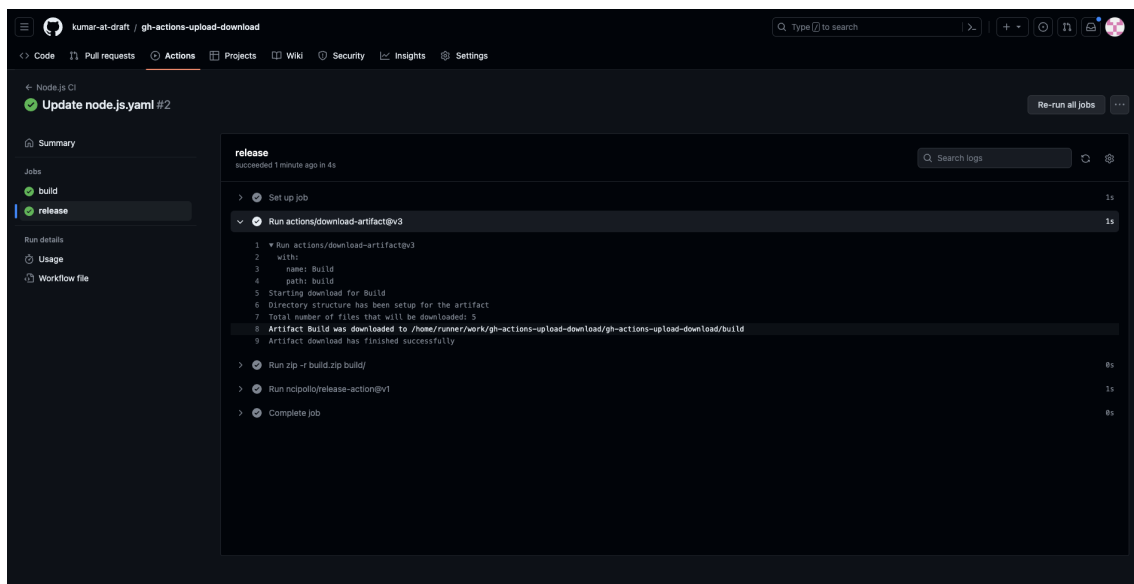


Figure 4: GitHub Actions Artifacts

8.4 Output

- HTML reports (`newman-report-*.html`) stored in `/reports`.

- Reports archived as GitHub Actions artifacts for download.

8.5 Benefits of CI Integration

- **Automation:** Consistent test execution on every code change.
- **Early Bug Detection:** Identify regressions immediately after commits/PRs.
- **Traceability:** HTML reports provide clear details for debugging.
- **Scalability:** Easily extend to more collections or environments (e.g., staging, production).

9 AI / LLM Support & Prompts

9.1 Tool Used

ChatGPT (AI/LLM):

- Generated draft test cases (boundary, validation, and security scenarios).
- Suggested bug report entries (Title, Defect Description, Steps, Severity/Priority).
- Provided Markdown templates for the final report structure and self-assessment table.
- Drafted GitHub Actions YAML for Newman/Postman CI integration.

9.2 How AI was Applied

- **Test Case Generation:** Suggested edge cases (invalid/empty email, password boundaries, DOB future/underage, SQLi/XSS payloads, RBAC violations).
- **Bug Report Drafting:** Converted failed test cases into bug report entries (Title, Defect Description, Steps, Severity/Priority).
- **Documentation:** Provided Markdown template for report sections, self-assessment table, and grading criteria.
- **CI Integration:** Drafted a GitHub Actions YAML workflow for Newman/Postman automation.

9.3 Prompts Used

- “Generate 30 detailed test cases for Sign in API (cover invalid email, weak password, token expiration).”
- “Create 30 test cases for Add User API including duplicate email, DOB validation, SQLi/XSS, RBAC checks.”
- “From failed test cases, draft bug reports with Bug ID, Steps, Expected, Actual, Severity, Priority.”
- “Write a Markdown report template for API Testing with sections: Intro, Environment, Methodology, APIs, CI, Summary, Self-Assessment.”
- “Provide GitHub Actions YAML to run Newman collections for Sign in, Add User, Edit User APIs.”

10 Self-Assessment Table

Criteria	Outcomes	Percent	Self-Assessed
1. API 1		30%	
1.1 Report	Clear, detailed report for Sign in API	15	15
1.2 Test Cases (30)	30 well-documented test cases with Preconditions, Steps, Expected/Actual	5	5
1.3 Screenshots on Testing Tools	Included Postman Runner and Newman execution logs	5	5
1.4 Bugs	All identified defects reported with Steps, Expected, Actual, Severity	5	5
2. API 2		30%	
2.1 Report	Detailed Add User API report with coverage and findings	15	15
2.2 Test Cases (30)	30+ test cases covering validation, security, RBAC, audit logging	5	5
2.3 Screenshots on Testing Tools	Screenshots of requests/responses and failed cases	5	5
2.4 Bugs	All Add User bugs logged with reproducible steps	5	5
3. API 3		30%	
3.1 Report	Comprehensive Edit User API report with evidence	15	15
3.2 Test Cases (30)	30 test cases with edge cases (duplicate email, DOB, RBAC, SQLi/XSS)	5	5
3.3 Screenshots on Testing Tools	Screenshots showing Postman execution results	5	5
3.4 Bugs	All discovered Edit User bugs documented	5	5
4. Formatting & Presentation	Well structured, readable, properly formatted report	10%	10
Total		100%	100